ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ _____

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ ИНТЕРНЕТА ВЕЩЕЙ

по курсу: Инфокоммуникационные системы и сети

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4329		Д.С. Шаповалова
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

1. Цель работы:

Приобретение навыков проектирования приложения интернета вещей и интерфейса для отображения состояния системы.

2. Задание:

1. В рамках предложенной (или своей) темы спроектируйте систему, состоящую из 4-5 уникальных классов. Подумайте имеют ли классы общие методы и свойства, если таковые есть, выделите их в абстрактный класс. Обозначьте связь элементов системы (Подсказка: не лишним будет создать отдельный объект, который не имеет физического воплощения, а отвечает только за обработку и передачу данных между другими объектами, если это необходимо, в моём примере – это *MainControlUnit*). Подумайте какие данные в системе будет хранить база данных, обозначьте её отдельной сущностью, связями обозначьте какие данные и с каких вещей она должна хранить. Поскольку следующие лабораторные подразумевают написание кода на Python, в котором модификаторы доступа носят рекомендательный характер, ими можно пренебречь.

2. В рамках предложенной (или своей) темы спроектируйте, как минимум два интерфейса. Два интерфейса могут быть нужны, например, для пользователя и администратора. На примере умного дома это может быть интерфейс ребёнка, который только отображает некоторые данные и позволяет управлять некоторыми из вещей и интерфейс родителя, которые может позволять более глубоко настраивать управление домом. Подумайте какие данные лучше скрыть от одного пользователя, какие лучше показать, как лучше их отобразить.

Выбранная тема – 3) Умная теплица.

3. Ход работы

Для выполнения лабораторной работы был использован редактор диаграмм Mermaid.

Была разработана иерархия классов, состоящая из шести основных сущностей. Центральным элементом диаграммы стал абстрактный класс Thing, который, был определён как базовый шаблон для всех устройств системы. В данном классе были декларированы общие атрибуты name и status, а также абстрактные методы connect() и emulate(), обязательные для переопределения в классах-потомках. Использование абстрактного класса позволило обеспечить единообразие интерфейсов взаимодействия с разнородными устройствами и реализовать принцип наследования объектно-ориентированного программирования.

От класса Thing были унаследованы два основных класса: Sensor и Actuator. Класс Sensor был дополнен атрибутами unit, value и last_update, а также методами get_value(), connect() и emulate() для получения, эмуляции и возврата показаний датчиков. Класс Actuator был расширен атрибутами power_state и mode, а также методами управления turn_on(), turn_off() и set_mode() для реализации логики исполнительных устройств, таких как системы полива, вентиляции и освещения.

Для реализации бизнес-логики автоматического управления климатом в систему был добавлен класс ClimateController, не являющийся цифровым двойником физического устройства, а представляющий собой управляющий модуль. В данном классе были определены атрибуты целевых и текущих значений температуры и влажности, а также методы check_conditions() и auto_control() для анализа показаний датчиков и формирования управляющих воздействий на исполнительные механизмы.

Координация взаимодействия всех компонентов системы была реализована посредством класса MainControlUnit, выполняющего функции центрального контроллера. В классе были предусмотрены атрибуты devices_count и system_status, а также методы collect_data(), process_data() и send_commands(), обеспечивающие сбор информации от датчиков, её обработку и рассылку команд исполнительным устройствам.

Для обеспечения долгосрочного хранения данных в диаграмму была включена сущность Database, представляющая документо-ориентированную базу данных MongoDB. В классе были определены методы save_sensor_data(), save_logs() и get_history() для сохранения показаний датчиков, системных логов и извлечения исторических данных для последующей визуализации.

Между классами были установлены ассоциативные связи с указанием кратности, отражающие логику потоков данных в системе. Множество объектов классов Sensor и

Actuator были связаны с единственным объектом MainControlUnit, что соответствует принципу централизованного управления.

Класс MainControlUnit был связан с ClimateController для передачи данных аналитики и с классом Database для сохранения информации.

Класс ClimateController был связан как с датчиками для чтения показаний, так и с исполнительными устройствами для отправки управляющих сигналов.

Кратность связей была указана в соответствии с реальной архитектурой: один контроллер обслуживает множество устройств, а каждое устройство взаимодействует с одним центральным узлом.

Полученная диаграмма классов представлена на рисунке 1:

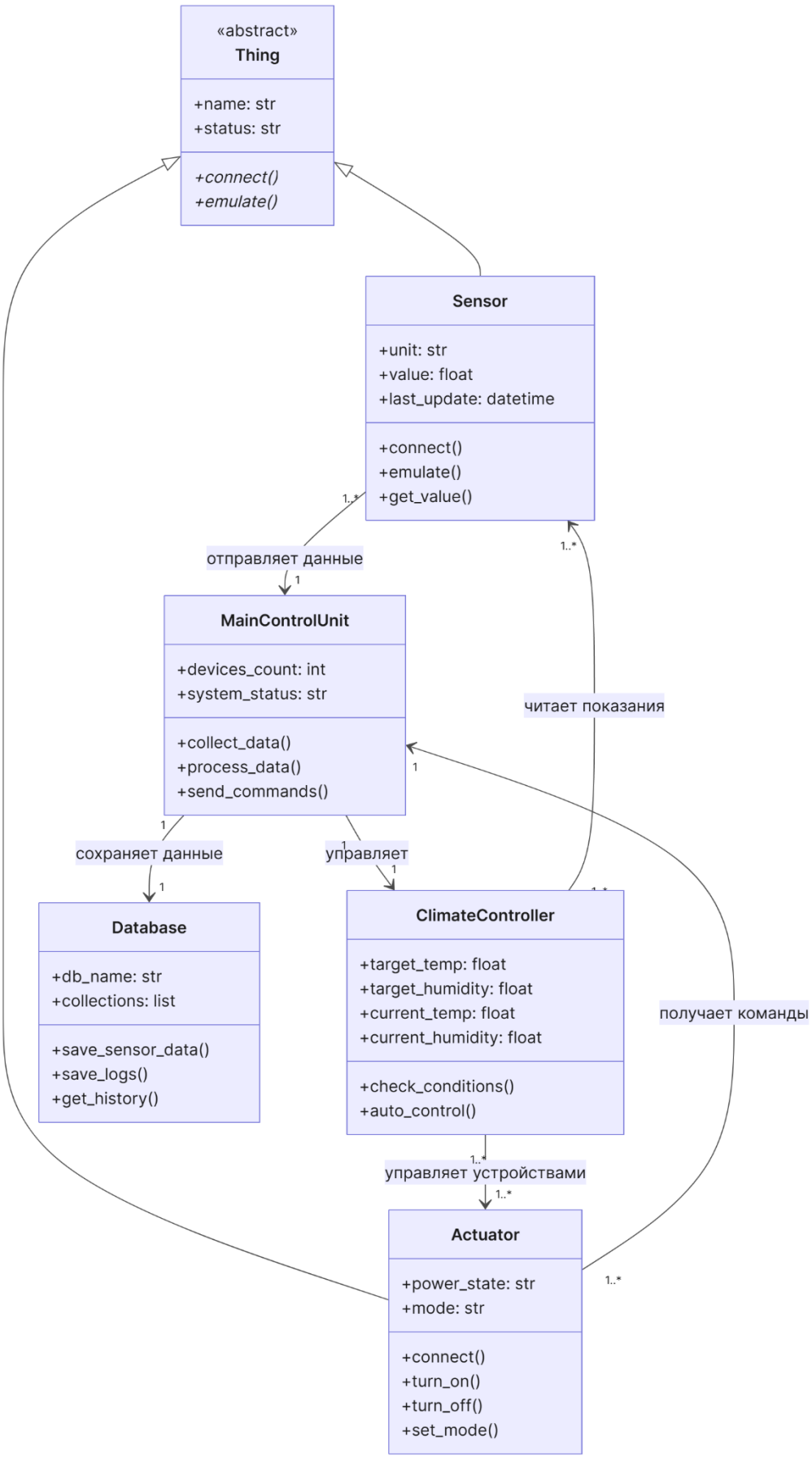

Рисунок 1 – Диаграмма классов

В рамках выполнения лабораторной работы было спроектировано два пользовательских интерфейса.

Интерфейс главной страницы предназначен для оперативного мониторинга состояния теплицы и базового управления. На странице отображаются текущие показатели микроклимата (температура, влажность, освещённость) и расположены кнопки активации автоматических режимов управления поливом, освещением и температурой.

Интерфейс настроек предоставляет расширенный функционал для администратора системы и включает четыре функциональных модуля: настройка характеристик системы, просмотр логов системы, расширенная аналитика и управление уведомлениями.

Оба интерфейса имеют единую навигационную панель с вкладками «Объекты», «Главная» и «Настройки», что обеспечивает удобную навигацию между разделами системы.

Интерфейсы разработаны в программе Figma и представлены на рисунке 2:

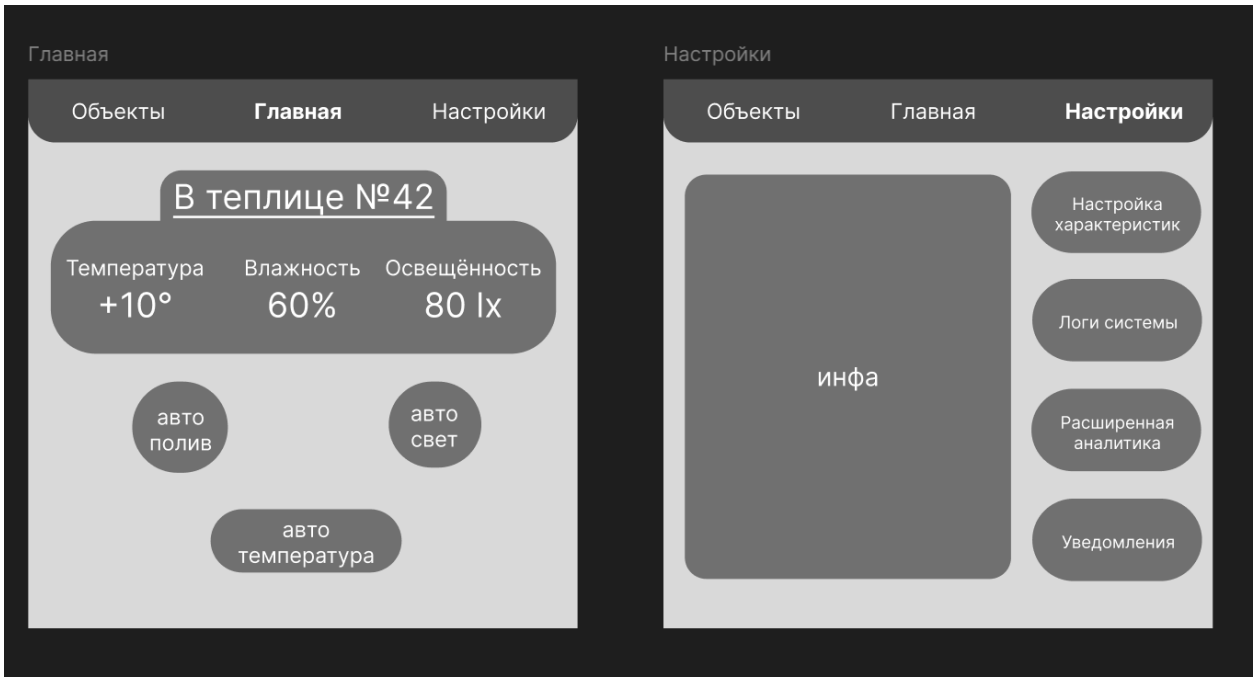

Рисунок 2 – Интерфейсы умной таблицы

ВЫВОД

В результате выполнения лабораторной работы были приобретены навыки проектирования приложения интернета вещей на примере системы «Умная теплица».

Спроектирована диаграмма классов UML, включающая абстрактный класс Thing, классы датчиков и исполнительных устройств, а также управляющие модули ClimateController и MainControlUnit.

Разработаны два пользовательских интерфейса – для оператора и администратора – с разграничением прав доступа и функционала.

Спроектированная архитектура соответствует трёхслойной модели IoT-систем (Physical Layer, Digital Twin Layer, User Interface Layer) и обеспечивает масштабируемость системы за счёт возможности добавления новых устройств без модификации базовой логики приложения.